

SAG Chronicles - Arhitectura de Ansamblu și Etapele de Dezvoltare

Cuprins

1. Context și Scope
2. Arhitectura C4 - Level 1: Context
3. Arhitectura C4 - Level 2: Container
4. Arhitectura C4 - Level 3: Component
5. Arhitectura C4 - Level 4: Code
6. Etapele de Dezvoltare
7. Decizii Arhitecturale

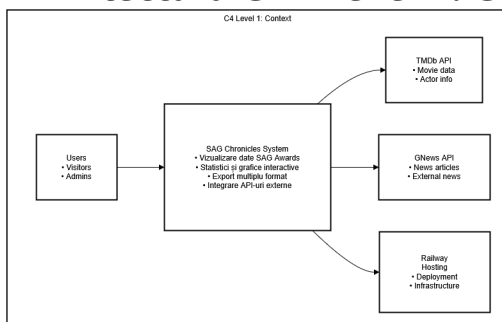
Context și Scope

SAG Chronicles este o aplicație web pentru vizualizarea flexibilă a datelor referitoare la nominalizările actorilor la Screen Actors Guild (SAG) Awards din ultimii ani. Aplicația oferă statistici, vizualizări și funcționalități de export în multiple formate.

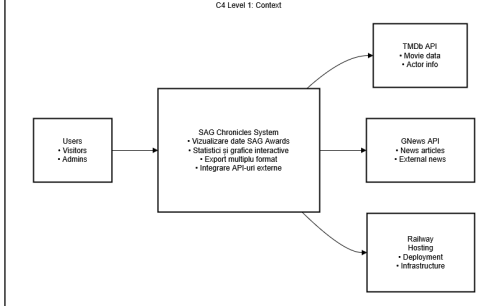
Obiective Principale

- Vizualizarea datelor SAG Awards în minimum 3 maniere diferite
- Export în formatele CSV, WebP și SVG
- Integrarea cu The Movie Database (TMDb) pentru informații suplimentare
- Suport pentru știri externe configurabile
- Interfață responsivă și modernă

Arhitectura C4 - Level 1: Context



Arhitectura C4 - Level 2: Container



Architettura C4 - Level 3: Component

Frontend Components

Frontend Container

Pages Layer

- index.html
- search.html
- top-actors.html
- category-stats.html
- winners-table.html
- admin.html

Styling Layer

- responsive styles
- globall.css
- admin.css
- category-stats.css
- fonts.css
- globals.css
- index.css
- search.css
- top-actors.css
- winners-table.css

Backend Components

Backend Container

Entry Point

- server.js

Routing Layer

- routes.js

Controllers Layer

- statsController.js
- searchController.js
- exportController.js
- adminController.js

Models Layer

- statsModel.js
- SearchMoviesModel.js
- GNewsModel.js
- adminModel.js

Data Layer

- setupDB.js
- importData.js
- actors_awards.db

JavaScript Utilities

- searchHelper.js
- topActorsHelper.js
- categoryStatsHelper.js
- winnersTableHelper.js
- exportHelper.js
- reportHelper.js

Configuration

- config.js
- package.json

Arhitectura C4 - Level 4: Code

Exemplu: Search Controller

javascript

```
// Controllers handle HTTP requests and coordinate between models
class SearchController {
  async handleCelebritySearch(req, res) {
    const query = req.query.query;
    try {
      // Coordinate with external API model
      const data = await SearchMoviesModel.fetchCelebrityNews(query);
      // Return structured response
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify(data));
    } catch (error) {
      // Error handling
      res.writeHead(500, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ error: 'Server error' }));
    }
  }
}
```

Exemplu: Database Model

javascript

```
// Models handle data operations and business logic
class StatsModel {
  static getTopActorsByNominations(callback) {
    const query = `
      SELECT ACT.name, COUNT(N.id) as nomination_count
      FROM Nominations N
      JOIN Actors ACT ON N.actor_id = ACT.id
      JOIN Awards A ON N.award_id = A.id
      WHERE A.year >= ?
      GROUP BY ACT.name
      ORDER BY nomination_count DESC
    `;
    db.all(query, [tenYearsAgo], callback);
  }
}
```

Etapele de Dezvoltare

Faza 1: Fundația și Infrastructura

Durată estimată: 1-2 săptămâni

Obiective

- Stabilirea arhitecturii de bază
- Configurarea environmentului de dezvoltare
- Crearea structurii de proiect

Activități

1. Setup Proiect

- Inițializarea repository-ului Git
- Configurarea package.json cu dependențele necesare
- Stabilirea structurii de foldere

2. Configurarea Serverului

- Implementarea server.js cu Node.js HTTP server
- Crearea sistemului de routing de bază
- Configurarea servingului de fișiere statice

3. Design Database

- Modelarea schemei de date (Actors, Movies, Categories, Awards, Nominations)
- Implementarea setupDB.js pentru crearea tabelelor
- Definirea relațiilor între entități

Faza 2: Backend Core și API-uri

Durată estimată: 2-3 săptămâni

Obiective

- Implementarea logicii de business
- Integrarea cu API-uri externe
- Crearea endpoint-urilor REST

Activități

1. **Models și Database Operations**

- statsModel.js - queries pentru statistici
- Implementarea CRUD operations

2. **External API Integration**

- SearchMoviesModel.js - integrare TMDB
- GNewsModel.js - integrare GNews
- Handling de erori

3. **Controllers și Business Logic**

- Implementarea statsController pentru endpoint-uri de statistici
- searchController pentru funcționalitatea de căutare
- exportController pentru funcționalitățile de export

4. **Data Import și Seed**

- importData.js pentru popularea inițială a bazei de date
- Scripturi pentru import din surse externe

Faza 3: Frontend Development

Durată estimată: 2-3 săptămâni

Obiective

- Crearea interfețelor utilizator
- Implementarea funcționalităților interactive
- Optimizarea pentru responsive design

Activități

1. HTML Structure și Layout

- index.html - landing page cu navigație
- search.html - interfața de căutare
- top-actors.html - statistici actori
- category-stats.html - grafice pe categorii
- winners-table.html - tabelul câștigătorilor
- admin.html - interfața administrativă

2. CSS Styling și Theming

- globals.css - stiluri de bază și tema dark
- Implementarea responsive design
- Stilizarea componentelor individuale

3. JavaScript Functionality

- searchHelper.js - logica de căutare și toggle movies/news
- Chart.js integration pentru vizualizări

Faza 4: Features Avansate și Optimizări

Durată estimată: 1-2 săptămâni

Obiective

- Implementarea funcționalităților avansate
- Testing și bug fixing

Activități

1. Advanced Features

- Admin panel (admin.html) cu funcționalități de management
- Sistem de raportare (reportHelper.js)
- Easter egg pentru accesul la admin (7 click-uri pe logo)

2. Export și Data Processing

- exportHelper.js - export în multiple formate
- Data transformation și formatting

3. Security și Validation

- Input validation și sanitization
- SQL injection prevention
- XSS protection

Faza 5: Deployment și Finalizare

Durată estimată: 1 săptămână

Obiective

- Deployment pe Railway
- Testing final și documentation

Activități

1. Deployment

- Configurarea Railway hosting
- DNS setup

2. Documentation și Testing

- Finalizarea README.md
- User acceptance testing

Decizii Arhitecturale

1. Tehnologii Alese

Component	Tehnologie	Justificare
Backend	Node.js	Este simplu de învățat și folosit pentru server-side development. Permite folosirea JavaScript-ului atât pe frontend cât și pe backend.
Database	SQLite	Este o bază de date simplă care nu necesită configurare complexă. Perfectă pentru proiecte mici și medii.
Frontend	Vanilla JavaScript	Nu necesită învățarea unui framework nou și oferă control total asupra codului.
Charts	Chart.js	Este o librărie populară și ușor de folosit pentru crearea de grafice interactive.
Hosting	Railway	Permite deploy rapid și simplu direct din Git.

2. Patterns Arhitecturale

MVC Pattern (Modified)

Models (Data Layer)

- Database operations (statsModel.js)
- External API integration (SearchMoviesModel.js, GNewsModel.js)
- Business logic encapsulation

Views (Presentation Layer)

- HTML templates (*.html)
- CSS styling (styles/*.css)
- Client-side JavaScript (utils/*.js)

Controllers (Logic Layer)

- Request handling (controllers/*.js)
- Data transformation
- Response formatting

Service Layer Pattern

- External API calls encapsulate în services
- Reutilizarea logicii între controllere

3. Securitate

Input Validation

```
javascript

// Exemplu din searchController
const query = req.query.query;
if (!query || typeof query !== 'string' || query.length > 100) {
    return res.status(400).json({ error: 'Invalid query parameter' });
}
```

SQL Injection Prevention

```
javascript

// Prepared statements în toate query-uriile
db.all(query, [sanitizedInput], callback);
```

XSS Protection

- Sanitizarea input-urilor în frontend
- Content-Type headers corecte
- Validarea datelor primite de la API-uri externe

5. Scalabilitate

Horizontal Scaling

- CDN pentru assets-uri statice

Vertical Scaling

- Database query optimization
- Efficient data structures

Concluzii

SAG Chronicles implementează o arhitectură robustă și scalabilă care respectă principiile moderne de dezvoltare web:

Puncte Forte

- Modularitate: Separarea clară a responsabilităților
- Maintainability: Cod bine organizat și documentat
- Performance: Optimizări pentru speed și user experience
- Security: Măsuri preventive implementate

Tehnologii Moderne

- REST API architecture
- Responsive web design
- Interactive data visualizations
- Multiple export formats
- External API integrations

Best Practices

- Separation of concerns
- DRY (Don't Repeat Yourself)
- Error handling comprehensiv
- Input validation și sanitization